Failure Analysis: Computer Science and the Therac-25

Jett Nguyen

ENC3246

University of Florida

10/13/2024

Table of Contents

Introduction *3*

1. Common Types of Failure in Computer Science *4*

1.1.1 Types and Causes of Failures 4

1.1.2. Bugs 4

1.1.3. Design Deficiencies 4

1.1.4. User Errors 5

1.2. Testing Types and Methods 5

1.2.1. Preventative Methods 6

1.2.2. Forensic Methods 6

1.2.3. Industry Standards 6

2. Case of Failure *8*

2.1. Case Description 8

2.2. Case Investigation 9

2.3. Recommendations 11

References *13*

Introduction

In the field of computer science, failure analysis is a vital aspect of ensuring that end products are efficient, functional, and safe. Examining past faults gives insightful glimpses into how and where these events transpired so that the future of innovation continues down an economically, sustainably, and ethically sound path. In the context of software engineering, certain protocols should be followed to ensure this during all steps of the design cycle process. However, there are various cases of failure that range from the early stages of computer science to the modern day as the digital landscape continues to take a solidified shape. This paper will discuss common failures, testing methods, and the malfunctions of the Therac-25 radiation therapy machine to demonstrate the significance of thorough failure analysis to minimize risks and catastrophic events.

1. Common Types of Failure in Computer Science

1.1.1 Types and Causes of Failures

Failure can be defined as the inability of a system to function correctly and efficiently within the scope of its desired abilities. The danger of software engineering is that the failure may not be apparent until its implementation. That is why programming faults must be caught before they are used. There are a few areas of error that must be inspected to mitigate risks.

1.1.2. Bugs

The most common and well-known problems in computer science are software bugs, which are lines of code that can cause a crash or unintended output. This happens during the development phase and takes the form of logical errors, memory leaks, and undefined conditions. They are so prevalent in programming because a simple line or character inaccuracy can cause them. While some bugs may only produce formatting issues, others lead to crashes, data corruption, or vulnerabilities in the security. On a human level, problems like these are practically unavoidable but must be caught early as projects become more complex in stages closer to publication.

1.1.3. Design Deficiencies

Adequate planning and establishing a solid framework for a system becomes important when examining how all components will unite to form one final product. The things that must be considered are what and how the different parts will interact. Compatibility issues are faced when something is not built to work with another structure, which can happen when dealing with various libraries and namespaces. Specific programs call for different syntax and formatting for references to other involved code. It is also important to recognize the intersection of software

and hardware because if lines control physical components, the software must codeswitch its instructions to that specific part. Failure to perform outlining can result in similar issues to bugs with the addition of inefficiency and difficulty fixing problems due to disorganization.

1.1.4. User Errors

Human errors like the ones mentioned previously originate during the development of software. However, the final users always have the potential to misuse the system if it is not built to handle incorrect operations. For example, invalid data types or hasty succession of programs are behaviors that can result in system failures. Instances like these can be prevented through both ends of the product life cycle. Developers can implement validation checks and better user interfaces, and users can receive proper training on the software. User errors remain a common problem despite the knowledge of their sources, so recognizing this type of failure is vital in solving software engineering issues.

1.2. Testing Types and Methods

In software engineering, various testing types are used to verify and assess the functionality of programs. One type of failure analysis testing is fault tree analysis. In this visualization, the top node correlates to an issue, which is then spread out to branches in a tree fashion to represent possible causes using logical gates. It is especially beneficial in programming to see how one code snippet can affect other code and ultimately lead to an undesirable event [1]. Within testing types like this, there are broadly categorized methods called preventative and forensic, which analyze pre- and post-failures, respectively. Each one can be followed by destructive and non-destructive means.

1.2.1. Preventative Methods

Preventative methods attempt to catch failures before the final product is released. The most utilized techniques in software engineering are non-destructive. One of those is dynamic testing, which involves using varying inputs and analyzing the potential bugs they may bring up to identify fault points. Errors may not be noticed until an invalid value is used and can cause reversible issues such as buffer overflows and memory problems that would have caused practical issues if it was in use [2]. An example of a destructive preventative method is stress testing, which can overload a system. The results of this may be destructive and damage the hardware involved in performing the test.

1.2.2. Forensic Methods

Forensic methods analyze how and where the failure occurred once it has already happened. In testing safely, log analysis is typically done in the aftermath of a failure, such as a result of a test mentioned above. This method can help build the fault tree to map out how the incorrect code led to the event since the log stores this information. Alternatively, when the failure caused complete destruction, and the diagnostics could not be found, it may be reproduced controllably. In a testing environment, following the failed steps gives insights into where the problem stemmed from in a system by protectively logging data.

1.2.3. Industry Standards

A couple of relevant industry standards in software engineering include IEEE 829 and ISO/IEC 12207. Firstly, IEEE 829 outlines the general format of how the testing process should be executed. This standard ensures proper testing and documentation of software, hardware, and

interfaces so that it is universally readable in other contexts of the field [3]. Next, ISO/IEC 12207 is the overarching goal of testing types and methods: sustainability. Managing software through its entire life cycle is imperative to innovation because engineers must know what happens with it before, during, and after its creation to develop reliably functional products. As shown in Fig 1., the cycle includes testing in the development stage to validation to post-production, where if flaws occur at any point, they can be fixed easily [4].

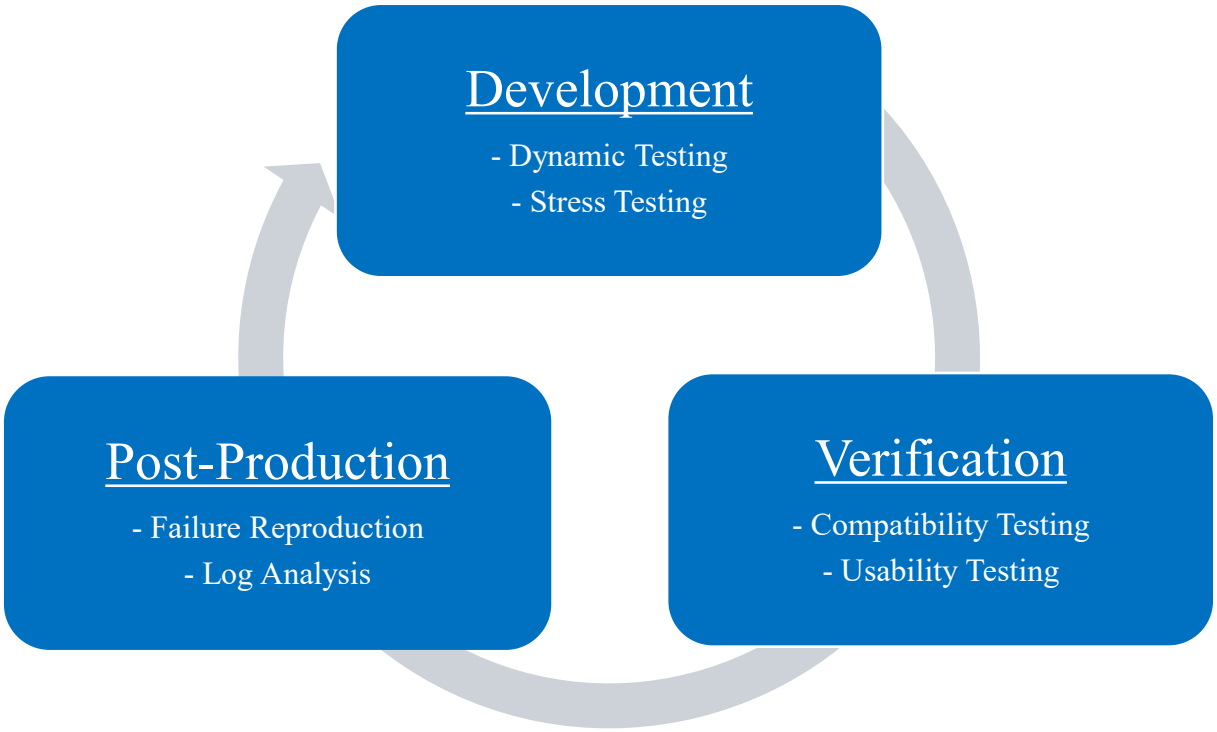

Fig. 1. ISO/IEC 12207 Testing Cycle

2. Case of Failure

2.1. Case Description

In the context of software engineering failure, the Therac-25 radiation therapy machine stands out as a historical landmark case. The partially automated state-of-the-art machine was developed to give cancer patients their exact dosages of radiation beams based on user input. By selecting options of high or low-energy beams, radiologists could ensure safe amounts based on the cancer severity. This method of treatment must be handled with care because radiation in higher applications can be fatal. However, due to it running on the foundation of underdeveloped and undertested programming, the use of the machine led to overdose deaths in a short period. Between 1985 and 1987, six cancer patients fell victim to the failure of the Therac-25. At the time, with computers having greater accuracy and less risk, the engineers decided for the machine to rely heavily on automation. This path proved too ambitious after the project's fatal results [5].

Software-reliant machines have become prevalent in the modern day and a way of the future. From 3D printers to washing machines, they are a part of humans' daily lives, directly or indirectly. But they did not get to this point without engineering hardships. Much like other automation, demonstration of proper testing and thoroughness are vital in medical device development where health is at risk. Through the process of failure analysis, the sources of error in the Therac-25 can be revealed to pave a safer and more reliable future for software.

2.2. Case Investigation

This failure of the Therac-25 stems from a lack of testing, ignorance of industry standards, and improper use. More specifically, a software error within the code of the medical device allowed for higher dosages than usual. The program design featured multiple settings for varying radiation levels, and the snippets that instructed the machine to run specific modes succumbed to race conditions. Switching the settings in a certain way could disturb the read-write process of mode states and result in value errors. When the switched tasks get into a situation like this, the flawed programming can cause them to run in parallel, desynchronize, and show conflicting machine states, as shown in Fig. 2 [6]. With simple, non-destructive testing methods, this problem could have been prevented during development. Even if it could not have been identified at this stage, more destructive testing, such as stress testing, could have made it apparent.

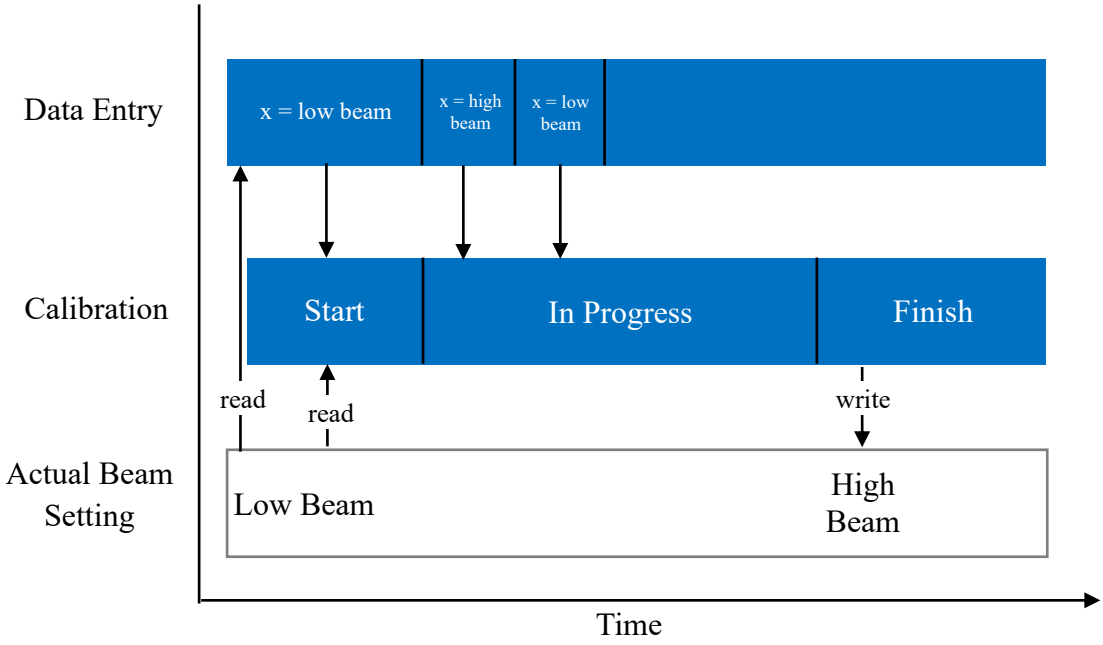

Fig. 2. Race conditions in the Therac-25

Within the fields of computer science and software engineering, preventative, non-destructive methods are utilized for testing. Most programming evaluations are completed before they are applied by debugging, simulating, peer-reviewing, and much more. As mentioned in the previous part, dynamic testing pertains to how this disaster could have been avoided. Attempting to input different combinations of inputs at varying speeds and successions could have revealed weak spots in the system. Even though the event occurred when this technology was primitive, early forms of non-destructive testing were around. Nonetheless, this case was a factor that made testing a more comprehensive process. Additionally, destructive testing might have been a more valuable choice when the cost on the other side is the lives of patients. After stress testing the machine under more complex conditions, it may break and show the vulnerabilities that may be exposed in implementation. Yet, the main problem lies not in the existing testing techniques but in the standards used at the time.

A couple of relevant industry standards in the case of the Therac-25 include IEEE 1012 and IEC 62304. Firstly, IEEE 1012 outlines the general validation of software so that it functions as intended with expected conditions. When testing, it must be considered that normal use may involve quickness in a fast-paced medical environment. It would be then that the flaw would come to light and be fixed beforehand. This standard is vital because it is a preventative measure that keeps the users safe when they use the product [7]. Similarly, IEC 62304 ensures that systems are safe and effective. The difference is that it encompasses medical software in its development stage from creation to exit, so even after the machine's initial failure, the problem should have been reassessed for its safety. Instead of continuing to employ the Therac-25 following the first incident, the integrity of it should be questioned and analyzed for failure [8].

Emphasizing thorough testing is important in any field, but adhering to standards like these is especially significant in computer science because of the longevity and reliability people expect from software.

2.3. Recommendations

Implications of failures such as these are health endangerment, user well-being, and software distrust. This machine has taken the lives of several people and dissolved the trust in software because of its deadly results. Multiple areas of improvement can be focused on in every sector of the product's life.

Firstly, referencing IEC 62304, the safety and integrity of medical software must be tested with methods such as some mentioned in part 1. If this were applied during the machine's development, measures would be taken to look for situations causing drastic undesirable outputs. The testing would not have to be done on patients, as the code could be simulated. Additionally, even after the rigorous pre-release procedures, the machine's performance should be monitored for changes in effectiveness or functionality. This pertains to IEEE 1012 because all aspects of the software's life cycle encompass its overall safety to continue to be trusted in treatment.

Second, considering both ethical and economic costs without exclusion should be a common practice in innovation. In this case, ethical costs were not appropriately weighed with the financial costs since extensive testing was not done. Additionally, expenses on lawsuits and repairs from fatal errors would be far more than what it would be to evaluate extreme

experiments. Ethical responsibilities must be fulfilled first before economic desires are taken into account.

Lastly, human error played a large role in the failure because the manners of operation allowed for it. While proper practices were not followed in creating the Therac-25, there should have been instructions for how it was intended to be used. Enforcing training or mandatory classes to provide knowledge on correct control is essential for new types of technology like this. Counteraction is the most effective technique when programming issues are still present in a system.

Through the application of these recommendations, failures like the Therac-25 overdoses can be avoided. In computer science, the implications of failures can be catastrophic, especially within the scope of medical device software development. Prevention is important because the digital consequences can take harmful physical forms. The specific benefit of the recommendations would be long-term software integrity and safety. By testing for undesired cases from planning to the release and providing training, the final users can be assured that the software they use will not cause any mild or life-threatening events.

References

[1] “Failure Analysis Methods and Tools - ALD Service,” *aldservice.com*. https://aldservice.com/Fracas/failure-analysis-methods-and-tools.html. [Accessed Sept. 9, 2024].

[2] M. A. Umar and C. Zhanfang, "A Comparative Study of Dynamic Software Testing Techniques," *International Journal of Advanced Networking and Applications,* vol. 12, *(3),* pp. 4575-4584, 2020. Available: https://login.lp.hscl.ufl.edu/login?url=https://www.proquest.com/scholarly-journals/comparative-study-dynamic-software-testing/docview/2564178735/se-2

[3] *IEEE Standard for System, Software, and Hardware Verification and Validation,* IEEE 829-2008, Jul. 18, 2008.

[4] *Systems and software engineering — Software life cycle processes*, ISO/IEC 12207:2017, Nov. 2017.

[5] N. Murphy, "Lock up your software: Part two," *Embedded Systems Programming,* vol. 14, *(1),* pp. 37-40, 2001. Available: https://login.lp.hscl.ufl.edu/login?url=https://www.proquest.com/trade-journals/lock-up-your-software-part-two/docview/218583174/se-2.

[6] P. A. McQuaid, "Software Disasters: What Have We Learned?" *Software Quality Professional,* vol. 11, *(3),* pp. 28-34, 2009. Available: https://login.lp.hscl.ufl.edu/login?url=https://www.proquest.com/scholarly-journals/software-disasters-what-have-we-learned/docview/214067219/se-2.

[7] *IEEE Standard for System, Software, and Hardware Verification and Validation,* IEEE 1012-2016, Sep. 29, 2017. [Online]. Available: https://standards.ieee.org/ieee/1012/5609/.

[8] Medical device software – Software life cycle processes, IEC 62304:2006, May 2006. [Online]. Available: https://www.iso.org/standard/38421.html.